

Delta Lake: Продвинутые техники

Advanced Level

Глубокое погружение во внутренний протокол Delta Lake, concurrency control, deletion vectors, CDC и кастомные реализации на Rust.

1. Транзакционный протокол Delta Lake

Протокол Delta Lake основан на optimistic concurrency control (OCC). Каждый коммит содержит атомарный набор действий (actions), которые либо применяются целиком, либо отклоняются при конфликте. Конфликт обнаруживается через проверку версии последнего коммита: если за время подготовки нового коммита кто-то другой уже записал новый коммит, текущий коммит отвергается и процесс повторяется с новым базовым снимком. Этот механизм гарантирует сериализуемую изоляцию.

1.1 Структура JSON-коммита

Каждый файл коммита в `_delta_log/` содержит JSON-массив действий. Ключевые типы действий определены в спецификации протокола и реализованы в `delta-rs` через соответствующие Rust-структуры с полной сериализацией/десериализацией через `serde`:

Action	Структура в Rust	Назначение
Protocol	Protocol	Минимальные версии reader/writer, включённые фичи
Metadata	TableMetadata	Схема таблицы, партиции, конфигурация, <code>createdTime</code>
Add	Add	Новый файл: <code>path</code> , <code>size</code> , <code>stats</code> , <code>partitionValues</code> , <code>deletionVector</code>
Remove	Remove	Удаление файла: <code>path</code> , <code>deletionTimestamp</code> , <code>dataChange</code>
CommitInfo	CommitInfo	Метаданные коммита: <code>operation</code> , <code>user</code> , <code>metrics</code> , <code>timestamp</code>
CDC	CDC	Change Data Feed запись для потоковой обработки
DomainMetadata	DomainMetadata	Метаданные сторонних систем (e.g. Iceberg compat)

1.2 Checkpoint и Log Compaction

Чекпоинт представляет собой Parquet-файл, содержащий все действия Add/Remove для текущей версии. Вместо чтения всех JSON-файлов от версии 0 до N, библиотека загружает последний чекпоинт и дочитывает только последующие JSON-коммиты. Файл `_last_checkpoint` содержит JSON с полями `version` и `size`, что позволяет мгновенно найти последний чекпоинт без сканирования директории.

По умолчанию чекпоинт создаётся каждые 10 коммитов (настраиваемый параметр). Лог-компакция объединяет множество JSON-файлов в один для более быстрой загрузки. В `delta-rs` эти операции доступны через `protocol::create_checkpoint()` и `protocol::cleanup_expired_logs_for()`. Важно: чекпоинтирование является атомарной операцией и не блокирует параллельные записи в таблицу.

1.3 Commit Strategies

Библиотека поддерживает две стратегии коммита. `CommitOrBytes::TmpCommit` использует атомарный `rename` (`copy-if-not-exists`) через временный файл. `CommitOrBytes::LogBytes` использует `conditional put` с проверкой версии. Выбор стратегии зависит от хранилища: для S3 используется `LogBytes` с `DynamoDB` для координации, для локальной FS - `TmpCommit` через атомарный `rename`. Каждый коммит также генерирует уникальный `operation_id` (UUID) для отслеживания и поддержки `CustomExecuteHandler`.

2. Механизмы конкурентного доступа

Delta Lake реализует несколько уровней блокировок для безопасной параллельной работы с таблицей. Основной механизм - оптимистичная concurrency control (OCC), но протокол также определяет правила разрешения конфликтов для различных операций.

2.1 Optimistic Concurrency Control

При OCC каждый читатель фиксирует версию таблицы, на которой он основывает свою работу. При попытке записи проверяется, что базовая версия не изменилась. Если изменилась - происходит конфликт, и операция повторяется с новой

базовой версией. В delta-rs этот механизм реализован в trait LogStore через метод write_commit_entry(), который возвращает TransactionError::ConcurrentModification при конфликте. Количество повторных попыток можно настроить через конфигурацию.

2.2 Правила разрешения конфликтов

Протокол Delta Lake определяет детальные правила для разрешения конфликтов между разными типами операций. Не все конфликты приводят к ошибке:

Операция A	Операция B	Результат
Append	Append	Обе успешны (безопасно)
Append	Optimize	Append перезапускается с новой версией
Optimize	Optimize	Вторая перезапускается с новой версией
MERGE	MERGE	Конфликт - вторая перезапускается
UPDATE	DELETE	Конфликт - вторая перезапускается
Write	Schema Change	Конфликт - схема изменилась
OPTIMIZE	Vacuum	Безопасно (независимые операции)

2.3 DynamoDB Commit Coordinator (S3)

При работе с Amazon S3 стандартный подход использует try-retry с conditional put. Однако это может привести к livelock при высокой конкурентности. Delta Lake поддерживает DynamoDB как external commit coordinator. DynamoDB обеспечивает атомарный compare-and-swap через ConditionalWrite, что гарантирует прогресс даже при высокой нагрузке. В delta-rs это настраивается через storage options с параметром dynamodb_locking_enabled=true.

3. Deletion Vectors (универсальный формат)

Deletion Vectors (DV) - это механизм, позволяющий удалять и обновлять строки на уровне файлов без перезаписи всего Parquet-файла. DV представляет собой компактный битовый массив (bitmap) или Roaring Bitmap, в котором установлены биты для удалённых строк. Физически DV хранится в отдельных файлах в директории таблицы и ссылается из Add action через поле deletionVector.

3.1 Как работают Deletion Vectors

Когда выполняется UPDATE или DELETE, вместо перезаписи Parquet-файла Delta Lake создаёт новый Deletion Vector. При чтении библиотека проверяет, есть ли у файла DV, и если есть - пропускает строки, отмеченные в DV. Это аналогично soft-delete, но на уровне storage без необходимости менять файл данных. Старый Parquet-файл остаётся на диске, но «потерянные» строки больше не видны при чтении.

3.2 Структура DeletionVectorDescriptor

В delta-rs DV описывается структурой DeletionVectorDescriptor с полями: storageType (формат хранения), pathOrInlineDv (путь к файлу или инлайн), offsetInFile (смещение в файле), sizeInBytes (размер), cardinality (количество удалённых строк). Поддерживаемые типы хранения: INLINE (для малых DV), FULL_FILE (отдельный файл) и PARTIAL_FILE (часть файла). Эта фича требует явного включения через table.add_feature() и требует минимальной версии writer 4.

3.3 Производительность

DV значительно снижает I/O при UPDATE/DELETE: вместо перезаписи всего файла (который может быть 256 MB+), записывается небольшой DV (обычно килобайты). Однако при чтении добавляется накладный расход на загрузку и применение DV. Рекомендуется регулярно запускать OPTIMIZE для объединения файлов с DV в чистые файлы без «дырок». Порог для автоматической компакции настраивается через таблицу конфигурации.

4. Change Data Feed (CDC)

Change Data Feed (CDF) - это функциональность Delta Lake, которая фиксирует все изменения в таблице (insert, update, delete) в виде потоковых записей. CDF позволяет строить пайплайны для синхронизации данных между системами, реализовать event sourcing и отслеживать lineage данных.

Для включения CDF необходимо установить table property delta.enableChangeDataFeed = true. После этого каждый коммит с dataChange=true генерирует CDC-файлы (Parquet), содержащие записи с колонками _change_type (insert/update/delete_preimage/update_postimage) и _commit_version. В delta-rs CDC-файлы читаются через отдельный API.

4.1 CDF в delta-rs

Библиотека delta-rs поддерживает чтение CDC через метод `table.cdf()` который возвращает `RecordBatch` с изменениями для заданного диапазона версий. Каждая запись содержит тип изменения, старые и новые значения. Это полезно для инкрементальной загрузки в downstream-системы, репликации данных и построения материализованных представлений.

```
// Чтение CDC между версиями 10 и 20
let changes = table.cdf("10", "20").await?;
for batch in changes {
  println!("Change type: {:?}", batch.column_by_name("_change_type"));
}
```

5. Row Tracking и Column Mapping

Row Tracking позволяет однозначно идентифицировать строки в таблице даже после операций MERGE, OPTIMIZE и других перестроек файлов. Каждая строка получает уникальный `row_id`, который сохраняется в Add action через поле `base_row_id`. Это критически важно для Deletion Vectors: при компактации файлов DV-ы перераспределяются, и `row_id` обеспечивает корректность маппинга.

Column Mapping (именованные колонки вместо позиционных) позволяет переименовывать и удалять колонки без физической перезаписи данных. Вместо порядкового номера колонки в Parquet-файле, маппинг использует уникальный `columnId` из метаданных таблицы. Это обеспечивает надёжную эволюцию схемы при совместной работе нескольких процессов с таблицей. Feature flag: `columnMappingMode = 'name'`.

6. Реализация собственной LogStore

Архитектура delta-rs позволяет реализовать собственные хранилища через два trait: `ObjectStoreFactory` и `LogStoreFactory`. `ObjectStoreFactory` создаёт экземпляр `object_store crate` для чтения/записи файлов данных. `LogStoreFactory` создаёт `LogStore` для управления транзакционным логом.

Для регистрации кастомного хранилища используется глобальный реестр на основе `DashMap` с ленивой инициализацией через `LazyLock`. При инициализации `LogStore` создаётся URI-схема, по которой фабрика определяется автоматически. Примеры: `s3:// -> S3LogStore`, `az:// -> AzureLogStore`, `gs:// -> GCSLogStore`. Для кастомной схемы (например, `mystore://`) необходимо зарегистрировать фабрику через `object_store_factories().insert()` и `logstore_factories().insert()`.

```
// Регистрация кастомного хранилища
use deltalake::logstore::logstore_factories;
use std::sync::Arc;

logstore_factories().insert(
  "mystore://".parse().unwrap(),
  Arc::new(MyCustomLogStoreFactory)
);
```

7. Интеграции и экосистема

Delta Lake интегрируется с множеством compute-движков и инструментов. delta-rs служит мостом между Rust-экосистемой и Delta Lake форматом.

Инструмент	Тип интеграции	Особенности
Apache DataFusion	TableProvider trait	SQL-запросы, DataFrame API, pushdown предикатов
Apache Arrow	Нативная поддержка	Zero-copy columnar, RecordBatch exchange
Polars	LazyFrame scan	Чтение через delta-rs -> Arrow -> Polars
DuckDB	Parquet чтение	Косвенная интеграция через файлы
Python (PyO3)	deltalake pip package	Полный API, доступен через PyArrow
Iceberg (UniForm)	Метаданные compat	Чтение Iceberg-совместимых Delta таблиц

Особый интерес представляет UniForm (Universal Format) - совместимость метаданных Delta Lake и Apache Iceberg. Это позволяет читать Delta-таблицы с помощью Iceberg-совместимых движков без конвертации. UniForm добавляет Iceberg-метаданные в Delta-чекпоинты, что делает таблицы доступными для обоих форматов одновременно.

8. Продвинутое тюнинг-производительности

8.1 Data Skipping

Delta Lake хранит min/max статистику по каждой колонке в Add action (поле stats). При сканировании таблицы библиотека пропускает файлы, чья статистика не совпадает с предикатом запроса. Для максимальной эффективности используйте Z-ORDER по часто фильтруемым колонкам и собирайте статистику DATA_COMPRESSION с настройкой `delta.dataSkippingStatsColumns`.

8.2 Caching (Foyer)

delta-rs поддерживает кэширование I/O через feature-flag `delta-cache` на базе библиотеки `foyer`. Кэш хранит часто используемые фрагменты файлов Parquet и JSON-коммитов в памяти с LRU-эвикцией. Это особенно эффективно при повторных запросах к одним и тем же партициям и при частом сканировании транзакционного лога. Настройка размера кэша через конфигурацию.

8.3 IO Runtime

Библиотека позволяет настроить собственный `tokio runtime` для I/O операций через `DeltaTableConfig.io_runtime`. Это полезно при интеграции в уже существующие приложения с собственным runtime. По умолчанию создаётся отдельный runtime с `multi-threaded scheduler`. Вы также можете задать `log_buffer_size` для контроля параллелизма storage-операций и `log_batch_size` для размера батча при обработке лога.

9. Вклад в развитие delta-rs

Архитектура `delta-rs` модульная и хорошо структурированная. Основные области для контрибьюшена: новые хранилища (LogStore implementations), операционные оптимизации, улучшение парсинга протокола и расширение DataFusion интеграции. Проект использует `workspace` с множеством crates - каждый в `crates/` директории. Тесты консолидированы в два бинарника (`it` и `it_datafusion`) для ускорения компиляции. CI работает на GitHub Actions с матрицей features.

Ключевые принципы кода: широкое использование trait-based абстракций, `Arc<dyn Trait>` для shared ownership, builder-pattern для API, `thiserror` для типобезопасной обработки ошибок, `serde` для сериализации протокола, и feature-gates для управления зависимостями. При добавлении нового хранилища необходимо реализовать `ObjectStoreFactory`, `LogStoreFactory` и зарегистрировать их через глобальные реестры.

1. Изучите `delta_kernel`: `delta-rs` делегирует парсинг протокола в официальный `delta_kernel` crate. Понимание kernel поможет при работе с actions и snapshots.
2. Используйте tracing: Библиотека использует crate tracing для структурированного логирования. Включите DEBUG уровень для понимания внутреннего поведения при troubleshoot.
3. Профилируйте storage: Основное узкое место - I/O хранилища. Используйте metrics для мониторинга latency storage-операций.
4. Тестируйте с concurrency: Напишите интеграционные тесты с параллельными записями, чтобы убедиться в корректности работы OCC.
5. Мониторьте checkpoint: Регулярно проверяйте состояние чекпоинтов. Пропущенный чекпоинт приводит к деградации загрузки таблицы.